POC de triage médical bilingue

Rapport technique et recommandations

OCR Projet 14 · 5 novembre 2026 · bilan final du prototype et écarts documentés

Je construis un prototype de triage français / anglais basé sur Qwen3-1.7B-Base. Ma démarche relie la préparation des données, deux étapes de SFT LoRA, un pilote DPO et une démonstration traçable. Je distingue les résultats techniques, l'accord avec des annotations de modèles et une validation clinique, qui n'a pas été réalisée.

Décision actuelle

Je conserve le SFT triage v3 comme candidat principal, sélectionné sur validation. Le pilote DPO améliore son objectif de préférence sans améliorer le classement d'urgence observé. La capacité de QA générale se dégrade fortement après la spécialisation ; je limite donc la démonstration au triage sur cas fictifs.

Résultats et preuves disponibles

Élément	Constat
Corpus SFT nettoyé	4 974 entrées, dont 3 986 pour l'entraînement
Triage de validation	Macro-F1 priorité 0,7715 ; rappel urgence maximale 95/100
Pilote DPO	256/64 paires ; 32 pas ; pas de gain de triage observé
Démonstration	Space Hugging Face ZeroGPU fonctionnel ; deux inférences réelles FR/EN
CI/CD	Tests, image Docker et déploiement Space réussis sur GitHub Actions
Test final	976 prédictions conservées ; SFT : macro-F1 priorité 0,7858, urgents 88/98
API vLLM	Trois cas fictifs sur A100 40 Go ; médiane HTTP 514 ms ; une revue humaine

Le test final et la vérification vLLM sont exécutés et leurs preuves récupérées. Deux limites de livraison subsistent : la démonstration publique utilise Transformers, et les poids finaux du pilote DPO sont absents de la sauvegarde Drive accessible. Je les documente sans revendiquer une conformité intégrale.

Trajet de lecture

Données et consensus → séparation des tâches → correction du protocole → comparaison SFT/DPO → architecture de démonstration → preuves et recommandations.

Les notebooks 01 à 06 suivent cette démarche. Les corpus et les poids restent hors de Git ; les scripts, contrats, protocoles et notebooks sont versionnés dans le dépôt privé.

1. Construire un corpus adapté à la tâche

Je commence par explorer chaque source. Une présence de contenu médical ne suffit pas à établir son utilité pour le triage ; je recherche une présentation disponible à l'accueil et une cible exploitable.

Source	Observation	Usage retenu
MediQAl, FR	QCM corrigés et cas ; CC BY 4.0 constatée	QA et candidats au triage
FrenchMedMCQA, FR	QCM ; licence non renseignée	Exploration, sans ajout au corpus final
MedQuAD, EN	Questions et réponses rédigées ; licence non renseignée	QA expérimentale, droits à clarifier
UltraMedical-Preference, EN	Préférences natives ; MIT constatée	DPO et candidats au triage

Aucune de ces sources ne fournit directement les étiquettes ESI nécessaires au projet. Je crée donc un corpus dérivé, avec une qualification temporelle et des annotations indépendantes de Codex, Claude et Mistral. Les schémas contrôlent les citations, les énumérations, les abstentions et la cohérence urgence / ESI.

Je retiens une majorité sur la qualification, la priorité puis l'ESI exact. Mistral intervient comme troisième vote ; pour UltraMedical, les appels sont ciblés sur les désaccords. Une majorité sur la priorité seule ne suffit pas pour une réponse cible qui contient un ESI précis.

Origine	Cas candidats	Cas uniques retenus	Entrées après traduction
MediQAl	2 007	700	1 400
UltraMedical	2 000	489	978

Les traductions permettent de présenter le même raisonnement dans les deux langues. Elles ne doublent pas le nombre de situations indépendantes. Le volume résulte de la qualité des candidats ; je ne relâche pas le consensus pour atteindre un quota.

Protection et provenance

L'audit automatique des 5 000 exemples initiaux ne détecte pas d'identifiant direct. Une revue humaine reste nécessaire : ce résultat ne constitue pas une preuve d'anonymisation complète ou de conformité RGPD. Les licences non renseignées doivent être clarifiées avant une diffusion des données dérivées. La démonstration est réservée aux cas fictifs.

Les annotations constituent un standard silver. L'indépendance des appels ne garantit pas l'indépendance des biais. Les cas exclus peuvent aussi rendre le jeu retenu plus facile que les situations rencontrées à l'accueil.

2. Deux apprentissages et des partitions figées

Je sépare l'apprentissage de réponses médicales générales et celui du triage. Les exemples partagent le schéma messages system / user / assistant, mais chaque phase conserve son objectif et ses partitions.

La préparation initiale produit 2 622 QA et 2 378 triages, soit 5 000 entrées et 50 % de chaque langue. L'audit ultérieur retire les prompts communs entre groupes et les cinq paires de test déjà exposées dans un pilote. Je conserve les versions historiques et je fige les partitions v3.

Corpus v3	Train	Validation	Test	Total
QA	2 098	262	262	2 622
Triage	1 888	238	226	2 352
Total SFT	3 986	500	488	4 974

Le total comprend les jeux d'évaluation : il ne désigne pas 4 974 exemples utilisés pour calculer les gradients. L'écart au volume initial est justifié par l'élimination des fuites. Les versions FR et EN d'un même cas restent dans le même split ; une exclusion s'applique à toute la paire.

Corpus DPO

Je conserve les préférences chosen/rejected natives d'UltraMedical, en excluant les sollicitations SFT et les chevauchements détectés. La version nettoyée compte 4 470 paires de train et 494 de validation. Le pilote en utilise 256 et 64, stratifiées par catégorie de préférence. Une paire du train dépasse la longueur autorisée et sort de l'éligibilité, sans troncature de la réponse.

Reproductibilité

Je conserve identifiants, groupes bilingues, provenance, empreintes SHA256, seed et manifeste d'exclusions. La version v3 ne réécrit pas les partitions historiques. Le script figer_partitions.py et ses tests vérifient les frontières. La comparaison finale doit utiliser les empreintes inscrites dans protocole_test_final.json.

Le volume équilibré entre langues est un choix de constitution. Il ne suffit pas à garantir une qualité équivalente en français et en anglais ; cette hypothèse est évaluée séparément.

3. Apprendre, diagnostiquer, corriger

Protocole SFT

J'adapte Qwen3-1.7B-Base, révision ea980cb0a6c2ae4b936e82123acc929f1cec04c1, avec LoRA. Le socle reste figé. Je supervise uniquement les tokens de la réponse assistant et je partage la construction du prompt avec l'inférence.

Les premiers essais montrent une incohérence entre le format d'entraînement et l'arrêt attendu. Je corrige le protocole avec le même préfixe et l'EOS natif du socle, identifiant 151643. Les exemples trop longs sont refusés plutôt que tronqués. Cette correction est vérifiée avant la poursuite des entraînements ; les sorties initiales restent conservées.

Le parcours corrigé est : socle → LoRA QA → poursuite du même LoRA sur le triage → pilote DPO. Le SFT triage v3 repart de la phase QA corrigée, après nettoyage des partitions.

Paramètre SFT triage v3	Valeur
Nombre de pas / batch effectif	357 / 16
Learning rate / warmup	0,0001 / 20 pas
Précision / seed	BF16 / 3407
Longueur maximale	1 024 tokens
Paramètres entraînables	17 432 576, environ 1 % du modèle chargé
Durée observée Colab	222 s, entraînement et validation finale

Je mesure aussi un rappel de QA pendant la spécialisation : 10 % des exemples du mélange, représentant environ 28 % des tokens de réponse. Cet essai met en évidence un compromis ; je n'en fais pas un remplacement automatique du triage seul.

Pilote DPO

Le DPO compare les réponses préférées et rejetées à une référence SFT figée. Désactiver le LoRA de référence utiliserait le socle et changerait l'expérience. Un test CPU miniature vérifie la mise à jour de la politique, les frontières de tokens et l'immobilité de la référence.

Le pilote utilise 32 pas, un batch effectif de 16, un learning rate de 0,000005 et beta = 0,1. Le checkpoint 32 est retenu par la loss de validation. La durée totale observée est de 100 s, dont 69 s d'optimisation. Ces durées sont propres à la session G4 ; elles ne constituent pas un benchmark matériel général.

4. Mesurer la tâche et ses effets secondaires

La macro-F1 des trois priorités constitue la mesure principale : elle traite chaque classe à poids égal. Je complète cette moyenne par le rappel des urgents, le sous-triage critique, l'accuracy ESI, les sorties analysables et la cohérence bilingue. La QA reste évaluée pour détecter l'oubli après spécialisation.

Le sous-triage critique désigne ici un ESI de référence 1 ou 2 prédit 3 à 5. Je distingue la proportion parmi les sorties analysables et celle parmi les urgents de référence. Une sortie non analysable reste une erreur de classification.

Validation comparée : 238 triages et 262 QA

Mesure	SFT v3	DPO pilote
Macro-F1 priorité	0,7715	0,7674
Accuracy ESI	0,7311	0,7269
Rappel urgence maximale	95/100	95/100
Sous-triages critiques	5, sur 3 cas distincts	Les mêmes 5
Accord bilingue ESI	88/119	91/119
QCM QA corrects	5/131	5/131
F1 lexical QA ouverte	0,0359	0,0363

La loss DPO de validation passe de 0,6931 à 0,5361 ; les préférences sont correctement ordonnées pour 51/64 paires. Cela ne se traduit pas par une amélioration du triage. Je conserve le SFT v3, sans interpréter le faible écart de macro-F1 comme une différence statistiquement établie.

La QA corrigée obtenait auparavant 59,54 % aux QCM et 0,3762 de F1 lexical. Sa dégradation après le triage empêche de présenter le modèle final comme un assistant médical généraliste. La relecture montre aussi des justifications introduisant une information absente, par exemple une fièvre : un format correct ne suffit pas.

Le SFT v3 obtient 0,8180 de macro-F1 en français et 0,7229 en anglais. Pour quantifier l'incertitude dans une étude ultérieure, je rééchantillonnerais les cas par paire, car les deux langues ne sont pas indépendantes.

4 bis. Test final après sélection

Le protocole est figé avant les calculs : 226 triages et 262 QA, décodage déterministe BF16, batch 32, EOS natif et limites de 192/256 tokens. Le candidat reste celui choisi sur validation. Je récupère les 976 prédictions (488 par modèle), vérifie leur ordre et recalcule les métriques localement : les résultats sont identiques aux rapports Colab. Le DPO n'est pas évalué sur ce test, car ses poids finaux sont absents de Drive.

Test figé	Socle	SFT triage v3
Macro-F1 priorité	0,0556	0,7858
Accuracy ESI	3,10 %	74,78 %
ESI analysables	10,62 %	100 %
Urgents retrouvés	0/98	88/98
Accord ESI FR/EN	Aucune paire doublement analysable	89/113
QCM corrects	1/131	0/131
F1 lexical QA ouverte	0,1376	0,0323

Le gain sur le socle inclut l'apprentissage du format de réponse. Je ne l'interprète pas comme une mesure isolée de connaissances médicales. Le SFT retrouve 88 urgents sur 98 : les 10 sous-triages critiques concernent 7 cas distincts. Ils représentent 10,20 % des urgents de référence, contre 5 % sur validation. La macro-F1 globale légèrement supérieure ne signifie donc pas une meilleure reconnaissance des situations urgentes.

La macro-F1 priorité est de 0,7804 en français et 0,7911 en anglais ; le rappel des urgents est respectivement de 42/49 et 46/49. Le classement relatif des langues change selon la partition. Les deux traductions d'un cas ne sont pas des observations indépendantes.

L'absence de réponse QCM correcte confirme la perte de la capacité QA après la spécialisation. Je garde le modèle comme démonstrateur de triage et je propose des adaptateurs séparés comme expérience future. Aucun réglage ni nouvel entraînement n'est choisi à partir de ce test.

5. Déployer un prototype traçable

Je sépare la collecte, le moteur d'inférence et l'audit. Le questionnaire demande les champs manquants et adapte une question à la présence lexicale de douleur. Il ne constitue pas un protocole clinique et ne délègue pas la conduite de l'entretien au LoRA. Le mode cas complet transmet directement la présentation saisie.

Deux voies techniques

Voie	Chaîne	Statut
Démonstration gratuite	Gradio → Transformers / PEFT → ZeroGPU	Fonctionnelle sur Hugging Face
API du POC	FastAPI → vLLM + LoRA → contrôle → SQLite	Vérifiée sur A100 40 Go, trois cas fictifs

Le sélecteur FR/EN traduit l'interface et réinitialise le cas. Les réglages exposent le mode guidé ou complet et les détails techniques ; les paramètres de génération restent fixés. Le même adaptateur est identifié par SHA256 avant publication.

L'API protège consultations et traces par une clé. Elle vérifie le modèle disponible, la longueur d'entrée et la cohérence ESI/priorité. Une sortie interrompue ou invalide conduit à une revue humaine sans priorité. Une panne du journal empêche la délivrance d'une proposition. Le contrat JSON simule une intégration SIH, sans connecteur hospitalier réel.

Les traces SQLite sont en clair et ne sont pas immuables ; la clé partagée n'identifie pas individuellement les soignants. Le Space propose une trace JSON par session, téléchargeable, avec nettoyage temporaire du cache. Ce mécanisme ne remplace pas un audit hospitalier persistant.

Vérifications réalisées

Les 47 tests locaux passent. Le premier run CI a exécuté 35 tests sur les données, métriques, API et Space, construit Docker et publié le paquet autorisé. Le test du conflit de port est ajouté à cette suite. Le run GitHub Actions 42074793162 est réussi : la révision attendue du Space est effectivement démarrée. Aucun corpus ni secret n'est envoyé dans ce paquet.

Deux parcours fictifs FR/EN ont produit une inférence réelle : 5,115 s et 3,870 s côté serveur, appel ZeroGPU inclus. Il s'agit de deux observations, pas d'un test de charge. Un essai ultérieur du mode direct a atteint le quota invité ; l'interface a présenté une indisponibilité sans priorité.

La vérification vLLM réelle sur A100 40 Go mesure 6,935 s, 0,385 s et 0,514 s par requête HTTP, soit une médiane de 0,514 s. Le premier appel comprend potentiellement des initialisations. Les trois traces SQLite concordent avec les réponses ; les contrôles HTTP renvoient 401 sans clé, 422 pour une entrée invalide et une question pour un cas incomplet.

Le cas anglais produit « delayed emergency » au lieu du libellé attendu « delayed urgency ». L'API retourne une revue humaine sans priorité. Cette observation montre une fragilité du contrat linguistique ; je ne relâche pas le contrôle après le test. Trois requêtes ne prouvent ni la capacité en charge ni la pertinence clinique.

La démonstration permanente utilise Transformers. L'essai vLLM est interne à une VM Colab temporaire, sans endpoint public persistant. Cet écart avec l'hébergement demandé reste explicite. Le conflit initial sur le port 8080 est corrigé en réservant un port libre transmis à Uvicorn ; un test CPU reproduit ce conflit.

6. Conservation, limites et recommandations

Conservation des expériences

Je récupère 363 fichiers Colab, soit environ 2,82 Go, avec contrôle des tailles et des SHA256. La copie MLflow locale contient 28 runs, dont 26 terminés et 2 échoués. Les URI d'artefacts sont adaptées dans une copie de la base ; les originaux restent conservés.

Les résultats du DPO sont disponibles, mais les derniers poids, le checkpoint 32 et la base après le DPO ne figuraient pas dans la sauvegarde distante accessible. La cause reste à établir. Je ne confonds pas un run observé avec la conservation complète de ses fichiers. La fermeture Colab attend maintenant la synchronisation Drive avant la libération de la VM.

Les quatre runs du test final sont récupérés dans une seconde copie MLflow locale. Les archives du test et de vLLM, les journaux et les traces SQLite sont conservés avec leurs empreintes. La nouvelle vérification Drive confirme l'absence des poids finaux DPO ; je conserve les résultats disponibles sans remplacer le checkpoint sélectionné par un autre.

Conditions et écarts restants

Niveau	Condition
Preuves techniques	Test final et vLLM vérifiés ; récupération finale DPO incomplète
Conformité au livrable cloud vLLM	Endpoint public vLLM persistant ou justification explicite de l'écart au cahier des charges
Données	Clarification des licences et revue humaine de l'anonymisation
Pertinence clinique	Évaluation indépendante par des professionnels, notamment des erreurs graves
Usage réel	Accès par rôle, journal sécurisé, supervision, protection des données et gouvernance

Roadmap proposée

1. Préserver les preuves finales et expliquer les écarts : sauvegarde DPO incomplète et hébergement public Transformers. Ne pas relancer un entraînement pour masquer un résultat défavorable du test.

2. Constituer une référence clinique indépendante : cas ambigus, urgences rares, informations absentes, négations et situations hors distribution.

3. Réduire l'oubli QA en comparant des adaptateurs distincts ou une stratégie multi-tâche ; mesurer chaque tâche à chaque étape.

4. Construire des préférences DPO réellement centrées sur le triage, revues cliniquement, puis comparer à budget et protocole constants.

5. Tester le système complet : questionnaire, explications, abstention, charge, latence et comportement lors des pannes.

6. Définir un pilote encadré avec validation humaine, seuils d'acceptation, surveillance et retour arrière avant toute extension.

Un modèle de 32B+ reste une piste de projection. Il ne corrige pas automatiquement les biais d'annotation, les fuites ou les défauts de protocole. Je comparerais ses bénéfices à ceux d'une meilleure référence et d'un apprentissage mieux ciblé.

Annexe. Retrouver les preuves et reproduire la démarche

Emplacement	Fonction
data_ingestion/01_preparation_donnees.ipynb	Exploration, qualification, consensus et corpus
evaluation/02_evaluation_initiale.ipynb	Socle, métriques et protocole
evaluation/03_execution_colab.ipynb	Corrections, SFT, DPO et choix sur validation
serving/04_poc_et_deploiement.ipynb	Questionnaire, API, audit et déploiement
evaluation/05_finalisation_colab.ipynb	Test figé, vérification vLLM et sauvegarde
evaluation/06_verification_vllm_colab.ipynb	Reprise vLLM sur port libre, sans réévaluer le modèle
reports/bilan_final_20261105.json	Synthèse des mesures et empreintes des preuves finales
data/processed/partitions_v3	Partitions nettoyées et manifeste d'exclusions
data/evaluation/colab_sft_dpo_v3	Prédictions et bilan de validation
data/colab_archive/original	Fichiers distants conservés sans réécriture
data/colab_archive/local	Copie MLflow avec URI d'artefacts locales

Je repars de la révision du socle et du hash de l'adaptateur inscrits dans serving/modele.json. Je ne substitue pas un checkpoint voisin à des poids manquants. La seed est conservée, mais ne garantit pas une identité numérique entre moteurs ou versions des bibliothèques.

Le script evaluation/restaurer_colab.py vérifie les transferts et adapte les chemins MLflow dans une copie. Les paramètres historiques restent inchangés. Le script evaluation/test_final.py vérifie les empreintes avant l'inférence, conserve les réponses JSONL et journalise les métriques et artefacts.

L'entraînement est séparé du déploiement : le workflow de publication vérifie les poids retenus, sans réentraîner le modèle. La récupération d'un paquet identique peut aboutir au même commit Hugging Face ; je vérifie la révision réellement servie.

Sources et provenance

Les notebooks 01 à 06, les manifestes de partitions, bilan_v3.json et les protocoles d'entraînement constituent les preuves locales. Les annotations proviennent de LLM ; la préparation du code et de la documentation est assistée par des outils d'IA. Je n'attribue pas à une revue clinique les contrôles automatiques effectués.

Références : docs.vllm.ai/en/v0.32.0/getting_started/installation/gpu/ ; huggingface.co/docs/hub/spaces-zerogpu ; huggingface.co/docs/hub/spaces-github-actions. Démonstration : huggingface.co/spaces/rayakevin/ocr14-triage-fr-en. CI : github.com/rayakevin/OCR_PROJET_14/actions/runs/42074793162.